

Analisis Performa Worker Thread dan Cluster pada Pemrosesan Paralel Node.js

Bagas Mahda Dhani¹, Aryo Pinandito²

Program Studi Teknologi Informasi, Fakultas Ilmu Komputer, Universitas Brawijaya
Email: ¹bagasmahda@student.ub.ac.id, ²aryo@ub.ac.id

Abstrak

Node.js dengan arsitektur *single-threaded* menghadapi tantangan dalam menangani tugas-tugas *CPU-bound*. Untuk mengatasi keterbatasan ini, Node.js menyediakan dua mekanisme paralelisasi bawaan, yaitu Worker Thread (*multi-threading*) dan Cluster (*multi-processing*). Penelitian ini bertujuan membandingkan performa kedua modul tersebut dalam menyelesaikan perkalian matriks sebagai representasi beban kerja *CPU-bound*. Metode eksperimen kuantitatif digunakan dengan empat ukuran matriks (512, 1024, 1536, dan 2048) dan 100 iterasi per skenario. Performa dievaluasi berdasarkan waktu eksekusi dan penggunaan memori. Data dianalisis menggunakan uji normalitas *Shapiro-Wilk* dan uji beda non-parametrik *Mann-Whitney*. Hasil menunjukkan bahwa Worker Thread lebih cepat pada beban ringan (512 dan 1024), sedangkan Cluster jauh lebih cepat pada beban berat (1536 dan 2048). Namun, Worker Thread secara konsisten jauh lebih hemat memori di semua skenario. Temuan ini mengungkap adanya *trade-off* antara kecepatan dan efisiensi memori, sehingga pemilihan modul harus disesuaikan dengan karakteristik beban kerja dan ketersediaan sumber daya sistem.

Kata kunci: Node.js, Pemrosesan Paralel, Worker Thread, Cluster, Performa, CPU-bound, Perkalian Matriks

Abstract

Node.js, with its *single-threaded* architecture, faces significant performance challenges when handling *CPU-bound* tasks such as intensive mathematical computations. To address this limitation, Node.js provides two built-in parallelization mechanisms: Worker Thread (*multi-threading*) and Cluster (*multi-processing*). This study aims to compare the performance of these two modules in executing matrix multiplication as a representative *CPU-bound* workload. A quantitative experimental method was employed using four matrix sizes (512, 1024, 1536, and 2048) with 100 iterations per scenario. Performance was evaluated based on execution time and memory usage. Data were analyzed using the Shapiro-Wilk normality test and the Mann-Whitney non-parametric test. Results show that Worker Thread is faster under light workloads (512 and 1024), whereas Cluster is significantly faster under heavy workloads (1536 and 2048). However, Worker Thread is consistently far more memory-efficient across all scenarios. These findings reveal a clear *trade-off* between speed and memory efficiency, indicating that the choice of module should be tailored to the specific workload characteristics and available system resources.

Keywords: Node.js, Parallel Processing, Worker Thread, Cluster, Performance, CPU-bound, Matrix Multiplication

1. PENDAHULUAN

Pesatnya perkembangan teknologi informasi dan pertumbuhan volume telah mendorong peningkatan kebutuhan akan sistem komputasi efisien. Fenomena ini menjadi sangat krusial, terutama dalam konteks aplikasi modern yang perlu mengelola dan menganalisis data dalam skala besar. Untuk menjawab tantangan

tersebut, aplikasi-aplikasi modern dituntut untuk menerapkan metode pemrosesan paralel. Pemrosesan paralel memegang peranan penting dalam mengoptimalkan penggunaan sumber daya komputasi dan memungkinkan eksekusi cepat terhadap aplikasi berbasis data yang besar (Dobre dan Xhafa, 2014).

Node.js, sebagai sebuah *runtime environment* untuk JavaScript, dikenal luas

karena arsitektur *non-blocking* I/O yang memungkinkan penanganan permintaan konkurensi tinggi. Namun, arsitektur ini menjadi kurang optimal saat menghadapi tugas-tugas yang bersifat *CPU-bound*, seperti perhitungan matematis kompleks. Node.js memerlukan implementasi teknik pemrosesan paralel yang memadai (Tilkov dan Vinoski, 2010).

Untuk mengatasi keterbatasan tersebut, Node.js menyediakan dua mekanisme utama untuk paralelisasi, yakni modul Worker Thread dan modul Cluster. Worker Thread menerapkan model *multi-threading*, di mana beberapa *thread* pekerja berjalan dalam satu proses Node.js. Sebaliknya, Cluster menerapkan model *multi-processing*, dimana setiap pekerja merupakan sebuah proses Node.js yang independen. Perbedaan ini berimplikasi pada bagaimana tugas didistribusikan, sumber daya dikelola, dan komunikasi antar unit paralel dilakukan, yang akan memengaruhi performa keseluruhan.

Penelitian sebelumnya oleh (Patrou et al., 2019) telah mengevaluasi pendekatan paralelisme di Node.js. Namun, penelitian tersebut memiliki dua keterbatasan utama. Penelitian tersebut terpaksa menggunakan library pihak ketiga, seperti WebWorker-Threads dan Napa.js karena modul Worker Thread bawaan Node.js belum stabil. Selain itu, kasus uji yang digunakan adalah estimasi *Monte-Carlo* yang dirancang untuk meminimalkan biaya komunikasi antar pekerja. Oleh karena itu, studi komparatif yang menganalisis modul bawaan Node.js terkini menggunakan kasus uji yang menguji biaya komunikasi masih sangat diperlukan untuk memberikan gambaran yang lengkap.

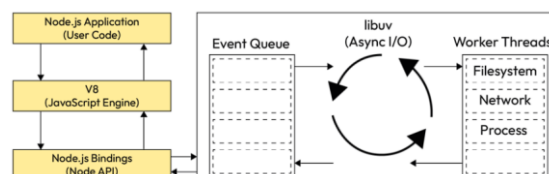
Oleh karena itu, penelitian ini bertujuan untuk melakukan evaluasi dan perbandingan performa secara komprehensif antara penggunaan modul Worker Thread dan modul Cluster dalam konteks pemrosesan paralel pada platform Node.js. Fokus evaluasi adalah pada efisiensi waktu eksekusi dan penggunaan memori ketika menangani tugas perkalian matriks dengan beberapa ukuran.

2. Tinjauan Pustaka

2.1. Node.js

Node.js adalah lingkungan *runtime* JavaScript yang dibangun di atas *engine* V8 Chrome yang memungkinkan eksekusi kode JavaScript di sisi server (Node.js, 2025b).

Node.js memiliki karakteristik *single-threaded* yang menggunakan arsitektur *event-driven* dan *non-blocking* *Input/Output*. Hal ini, memungkinkan Node.js menangani banyak permintaan bersamaan dengan efisien. Gambaran arsitektur Node.js yang memiliki karakteristik *single-threaded* bisa dilihat pada Gambar 1.



Gambar 1. Arsitektur Node.js
Sumber: Gascón (2024)

Node.js terdiri dari beberapa komponen, yaitu *V8 Engine*, *Node.js Bindings*, dan *libuv*. *V8 Engine* berfungsi mengeksekusi kode JavaScript dengan performa tinggi, sementara *Node.js Bindings* menjadi jembatan antara kode JavaScript dan modul native yang ditulis dalam bahasa C/C++. *Libuv* merupakan pustaka C yang mengelola operasi I/O secara asinkron dengan menggunakan *thread pool* di latar belakang (Gascón, 2024).

Namun, arsitektur Node.js yang bersifat *single-threaded* mengalami kendala saat menangani tugas *CPU-bound*. Tugas komputasi intensif yang dapat memblokir *event loop* dan membuat aplikasi tidak responsif. Untuk mengatasi keterbatasan ini, Node.js menyediakan modul bawaan seperti Worker Thread dan Cluster yang dapat melakukan mekanisme pemrosesan paralel.

2.2. Cluster

Cluster merupakan modul bawaan Node.js yang memungkinkan pembuatan beberapa proses Node.js yang dapat mendistribusikan beban kerja di antara proses-proses tersebut (Node.js, 2025a). Hal tersebut dicapai melalui model *multi-processing*, di mana setiap pekerja adalah proses Node.js yang independen.

komunikasi antara proses utama dan proses pekerja dilakukan melalui *channel Inter-Process Communication* (IPC). *Channel IPC* merupakan saluran komunikasi yang secara otomatis dibuat saat instance proses pekerja baru melalui metode *fork*.

2.3. Worker Thread

Worker Thread merupakan modul bawaan Node.js yang memungkinkan penggunaan *thread* untuk mengeksekusi kode JavaScript secara paralel (Node.js, 2025c). Modul ini menerapkan model *multi-threading*, di mana setiap *thread* pekerja berjalan dalam satu proses Node.js yang sama dengan *thread* utama.

Komunikasi antara *thread* utama dan *thread* pekerja dapat dilakukan melalui dua mekanisme. Pertama, pengiriman pesan menggunakan *MessagePort* bawaan yang secara otomatis terhubung saat *thread* pekerja dibuat. Metode ini menggunakan *Structured Clone Algorithm* untuk membuat salinan dari data sebelum mengirimkannya. Kedua, berbagi memori langsung menggunakan *SharedArrayBuffer* yang memungkinkan beberapa *thread* pekerja untuk membaca dan menulis ke memori yang sama secara bersamaan.

2.4. Perkalian Matriks

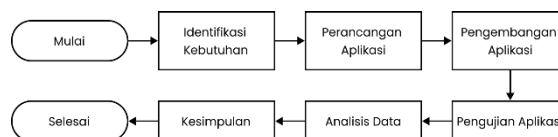
Penelitian ini menggunakan pendekatan konvensional perkalian operasi dua matriks persegi. Pendekatan ini melibatkan tiga lapisan perulangan bersarang yang masing-masing mengakses elemen dari baris dan kolom matriks, sehingga menghasilkan kompleksitas waktu sebesar $O(n^3)$ (Cormen et al., 2022). Secara matematis, perkalian operasi dua matriks persegi didefinisikan pada Persamaan 1.

$$f_{ij} = \sum_{k=1}^n x_{ik}y_{kj}, \quad 1 \leq i, j \leq n \quad (1)$$

Sifat dari perkalian matriks konvensional dapat diimplementasikan dalam pemrosesan paralel. Hal ini dikarenakan sifat independensi komputasi, di mana perhitungan untuk setiap elemen hasil pada matriks hasil sepenuhnya independen dan tidak bergantung pada hasil perhitungan elemen lainnya.

3. Metodologi Penelitian

Penelitian ini memiliki beberapa tahapan yang akan dilaksanakan. Tahapan penelitian terdiri dari identifikasi kebutuhan, perancangan aplikasi, pengembangan aplikasi, pengujian aplikasi, analisis data, dan kesimpulan. Seluruh tahapan ini dirancang sebagai acuan dalam pelaksanaan penelitian ini. Alur penelitian dapat dilihat pada Gambar 2.



Gambar 2. Diagram alur penelitian

3.1. Identifikasi Kebutuhan

Untuk memenuhi tujuan penelitian ini, kebutuhan fungsional utama sistem diidentifikasi. Sistem yang dikembangkan harus mampu melakukan operasi perkalian matriks dalam beberapa ukuran matriks besar menggunakan kedua modul paralel bawaan Node.js. Sistem juga harus dapat mengukur metrik performa utama, yaitu waktu eksekusi dan penggunaan memori secara akurat.

Sistem dikembangkan menggunakan bahasa pemrograman JavaScript pada lingkungan *runtime* Node.js versi 19.9.0. Proses pengembangan menggunakan *Integrated Development Environment* (IDE) Visual Studio Code dan dijalankan pada sistem operasi Windows. Seluruh pengujian dilaksanakan pada perangkat keras, yaitu laptop Lenovo Ideapad 3 dengan spesifikasi prosesor AMD Ryzen 7 5700U, memori 8 GB DDR4 RAM, dan penyimpanan 512 GB SSD.

3.2. Perancangan Aplikasi

Arsitektur aplikasi dengan pendekatan pemrosesan paralel yang terstruktur dan identik untuk kedua modul. Alur kerja aplikasi dirancang di mana *thread* atau proses utama bertanggung jawab untuk *instance* modul, membuat Matriks A dan B, memotong Matriks A menjadi beberapa bagian, dan mendistribusikan potongan Matriks A beserta salinan utuh Matriks B ke setiap pekerja. Setiap *thread* atau proses pekerja kemudian mengeksekusi perkalian matriks secara independen dan mengirimkan hasil parsialnya kembali ke *thread* atau proses utama, yang kemudian menggabungkan hasil akhir.

3.3. Pengembangan Aplikasi

Dua implementasi aplikasi dikembangkan secara terpisah namun dengan fungsionalitas yang sama. Implementasi pertama menggunakan modul Worker Thread, dan implementasi kedua menggunakan modul Cluster.

Implementasi Worker Thread akan menggunakan metode pengiriman pesan standar yang memanfaatkan *Structured Clone*

Algorithm, alih-alih mekanisme berbagi memori menggunakan *SharedArrayBuffer*. Pilihan ini diambil untuk membuat dasar perbandingan yang relevan dan setara secara mekanisme dengan modul Cluster. Modul Cluster juga beroperasi pada mekanisme yang sama, yaitu pengiriman data melalui IPC, bukan berbagi memori.

Dengan membandingkan performa *default* dari kedua implementasi ini, penelitian dapat secara efektif menganalisis bagaimana masing-masing mekanisme komunikasi bawaan dalam menangani beban pengiriman data yang dapat memengaruhi performa.

Untuk mendukung proses pengujian dan analisis data, implementasi di setiap modul juga menyertakan mekanisme pencatatan metrik performa waktu eksekusi total dan memantau penggunaan memori selama program dijalankan.

3.4. Pengujian Aplikasi

Pengujian akan dilaksanakan dengan mendefinisikan beberapa skenario beban kerja *CPU-bound* yang bervariasi. Skenario tersebut untuk mengevaluasi performa yang terdiri dari beberapa ukuran, yaitu 512, 1024, 1536, dan 2048. Untuk menjamin validitas dan reliabilitas data, setiap skenario pengujian akan dieksekusi secara berulang hingga terkumpul 100 sampel data untuk setiap modul.

Untuk metrik waktu eksekusi, pengukuran waktu ini diimplementasikan dengan menggunakan fungsi *Date.now()*. Perhitungan waktu akan dimulai tepat sebelum *thread* atau proses utama mendistribusikan bagian matriks ke *thread* atau proses pekerja. Perhitungan waktu kemudian dihentikan setelah *thread* atau proses utama menerima dan menggabungkan seluruh hasil parsial dari setiap *thread* atau proses pekerja.

Untuk metrik penggunaan memori, pengukuran memori diukur menggunakan *process.memoryUsage().rss*. RSS atau *Resident Set Size* merepresentasikan total alokasi memori fisik atau RAM yang digunakan oleh sebuah proses Node.js.

Terdapat perbedaan dalam pengukuran memori antara kedua modul. Pada implementasi Cluster, setiap proses pekerja akan mengukur penggunaan memori dan mengirimkan nilai RSS ke proses utama setelah komputasi selesai. Proses utama juga memantau penggunaan RSS secara periodik untuk mendapatkan nilai tertinggi. Total penggunaan memori untuk

Cluster kemudian dihitung dengan menjumlahkan nilai RSS tertinggi dari proses utama dengan nilai RSS dari setiap proses pekerja. Sebaliknya, pada implementasi Worker Thread cukup dilakukan secara periodik pada *thread* utama saja, karena nilai RSS pada *thread* utama sudah merepresentasikan total penggunaan memori dari keseluruhan *thread*.

3.5. Analisis Data

Data hasil pengujian dianalisis secara statistik menggunakan perangkat lunak RStudio. Proses analisis data dilakukan dengan memperhatikan uji asumsi klasik sebagai langkah awal sebelum pemilihan uji statistik parametrik atau non-parametrik. Uji asumsi klasik yang digunakan yaitu uji normalitas. Pada uji ini menggunakan *Shapiro-Wilk* di mana nilai *p-value* > 0,05 menandakan data terdistribusi normal. Jika asumsi tersebut terpenuhi, maka uji parametrik akan dilakukan menggunakan uji t. Namun, jika asumsi tidak terpenuhi, maka akan dilakukan uji non-parametrik menggunakan uji *Mann-Whitney*.

3.6. Kesimpulan

Pada tahap ini, hasil dari analisis data akan dijelaskan secara lebih detail. Hasil uji t atau *Mann-Whitney* akan digunakan untuk mengidentifikasi metode paralel Node.js mana yang memiliki performa lebih baik berdasarkan waktu eksekusi dan penggunaan memori antara Worker Thread dan Cluster dalam menyelesaikan perkalian matriks. Kesimpulan akan ditarik dari hasil analisis data untuk menjawab hipotesis dan rumusan masalah yang disampaikan pada bagian awal penelitian. Kesimpulan juga berisi saran dan perbaikan untuk penelitian lebih lanjut.

4. Hasil dan Pembahasan

4.1. Hasil Pengambilan Data

Pada Bab ini menyajikan rangkuman data hasil pengujian dalam bentuk statistik deskriptif. Tabel-tabel berikut mencakup metrik waktu eksekusi dalam milidetik dan penggunaan memori dalam *megabyte*. Tabel mencakup ukuran-ukuran statistik seperti rata-rata (*mean*), nilai minimum, median, nilai maksimum, dan standar deviasi.

Tabel 1. Data statistik deskriptif Cluster

Metric	Size	Mean	Min	Median	Max	SD
--------	------	------	-----	--------	-----	----

	512	1281,98	1254	1281	1316	9,22
Waktu	1024	5263,88	5009	5216,5	6179	204,54
	1536	13958,44	12930	13962	14999	374,86
	2048	30780,29	29382	30843	33068	649,3
	512	1118,15	1112,75	1119,97	1122,31	3,31
Memori	1024	2677,25	2675,21	2677,19	2679,19	0,8
	1536	4649,84	4639,71	4649,9	4651,7	1,34
	2048	7295,14	7082,58	7305,21	7307,28	32,93

Pada modul Cluster yang ditunjukkan pada Tabel 1, waktu eksekusi dan penggunaan memori meningkat secara signifikan seiring dengan bertambahnya ukuran matriks. Waktu eksekusi meningkat dari 1.281,98 milidetik pada matriks 512 menjadi 30.780,29 milidetik pada matriks 2048, dengan standar deviasi yang meningkat yang menunjukkan variabilitas waktu yang meningkat pada beban kerja yang lebih besar. Sementara itu, penggunaan memori juga meningkat dari 1.118,15 MB pada matriks 512 menjadi 7.295,14 MB pada matriks 2048, tetapi standar deviasi yang relatif kecil sepanjang skenario, yang menunjukkan bahwa penggunaan memori cenderung stabil dan konsisten meskipun beban komputasi meningkat.

Tabel 2. Data statistik deskriptif Worker Thread

Metric	Size	Mean	Min	Median	Max	SD
Waktu	512	812,88	784	811	852	9,36
	1024	5074,44	4813	5069	5315	112,12
	1536	26707,33	24987	26563	28319	714,63
	2048	117501,73	113984	117152,5	128331	3113,84
Memori	512	356,18	315,51	355,82	392,09	13,76
	1024	1132,24	1131,23	1132,1	1134,62	0,64
	1536	1673,79	1671,9	1673,82	1676,3	0,89
	2048	2390,04	2370,98	2389,05	2411,54	8,35

Pada modul Worker Thread yang ditunjukkan pada Tabel 2, waktu eksekusi dan penggunaan memori juga meningkat seiring dengan bertambahnya ukuran matriks. Waktu eksekusi meningkat secara signifikan dari 812,88 milidetik pada matriks 512 menjadi 117501,73 milidetik pada matriks 2048, disertai kenaikan standar deviasi yang mengindikasikan tingginya variabilitas performa pada beban kerja berat. Sementara itu, penggunaan memori juga meningkat dari 356,18 MB pada matriks 512 menjadi 2390,04 MB pada matriks 2048 dengan standar deviasi yang tetap rendah, menandakan bahwa efisiensi memori tetap stabil dan konsisten meskipun beban komputasi bertambah.

4.2. Analisis Data

4.2.1 Uji Normalitas

Bagian ini menjelaskan hasil pengujian normalitas data yang telah dilaksanakan menggunakan uji *Shapiro-Wilk*. Pengambilan kesimpulan didasarkan pada perbandingan nilai *p-value* dengan tingkat signifikansi 0,05. Data

dinyatakan tidak terdistribusi normal jika *p-value* < 0,05, dan sebaliknya.

Tabel 3. Uji normalitas metrik waktu eksekusi

Modul	n	p-value	Kesimpulan
Cluster	512	0,06667	Tidak Terdistribusi Normal
	1024	8,957E-10	Tidak Terdistribusi Normal
	1536	0,01478	Tidak Terdistribusi Normal
	2048	0,1237	Terdistribusi Normal
Worker Thread	512	0,001398	Tidak Terdistribusi Normal
	1024	0,05819	Terdistribusi Normal
	1536	0,6193	Terdistribusi Normal
	2048	1,219E-09	Tidak Terdistribusi Normal

Tabel 3 menyajikan hasil uji normalitas untuk metrik waktu eksekusi di kedua modul. Untuk modul Cluster, data gagal memenuhi asumsi normalitas pada sebagian besar skenario dan pengecualian hanya terjadi pada ukuran 2048, di mana data terdistribusi normal. Di sisi lain, modul Worker Thread justru menunjukkan distribusi normal pada skenario ukuran menengah, yaitu 1024 dan 1536. Namun, data Worker Thread terbukti tidak terdistribusi normal pada ukuran 512 dan 2048.

Tabel 4. Uji normalitas metrik penggunaan memori

Modul	n	p-value	Kesimpulan
Cluster	512	1,724E-10	Tidak Terdistribusi Normal
	1024	0,6387	Terdistribusi Normal
	1536	2,642E-13	Tidak Terdistribusi Normal
	2048	2,2E-16	Tidak Terdistribusi Normal
Worker Thread	512	0,1472	Terdistribusi Normal
	1024	1,535E-07	Tidak Terdistribusi Normal
	1536	0,4574	Terdistribusi Normal
	2048	0,005369	Tidak Terdistribusi Normal

Tabel 4 menyajikan hasil uji normalitas untuk metrik penggunaan memori di kedua modul. Modul Cluster hanya menunjukkan distribusi normal pada satu skenario, yaitu ukuran 1024, dan gagal memenuhi asumsi normalitas pada tiga ukuran lainnya. Modul Worker Thread menunjukkan hasil yang bervariasi, dengan data terdistribusi normal pada ukuran 512 dan 1536, namun terbukti tidak terdistribusi normal pada ukuran 1024 dan 2048.

Berdasarkan hasil uji normalitas tersebut, dapat disimpulkan bahwa asumsi normalitas tidak terpenuhi secara konsisten di seluruh pasangan data yang akan dibandingkan. Oleh karena itu, metode non-parametrik menggunakan uji *Mann-Whitney* diputuskan sebagai metode yang paling sesuai untuk mengevaluasi uji beda pada penelitian ini guna memastikan validitas analisis.

4.2.2 Uji Beda

Bagian ini memaparkan hasil uji beda yang dilaksanakan untuk mengetahui apakah terdapat

perbedaan performa yang signifikan secara statistik antara modul Cluster dan Worker Thread. Uji ini dilaksanakan dengan menggunakan uji *Mann-Whitney* sebagai metode non-parametrik. Tingkat signifikansi yang digunakan untuk pengujian ini adalah 0,05. Dengan demikian, perbedaan performa dianggap signifikan secara statistik apabila nilai *p-value* yang diperoleh $< 0,05$.

Tabel 5. Uji beda metrik waktu eksekusi

n	p-value	Kesimpulan
512	2,2E-16	Terdapat perbedaan signifikan
1024	1,395E-14	Terdapat perbedaan signifikan
1536	2,2E-16	Terdapat perbedaan signifikan
2048	2,2E-16	Terdapat perbedaan signifikan

Tabel 5 menunjukkan bahwa nilai *p-value* untuk seluruh skenario ukuran matriks berada jauh di bawah ambang batas 0,05. Temuan ini mengindikasikan bahwa terdapat perbedaan yang sangat signifikan secara statistik dalam hal waktu eksekusi antara kedua modul di semua tingkat beban. Perbedaan signifikan ini menyimpulkan bahwa keunggulan performa Worker Thread pada beban kerja ringan di ukuran matriks 512 dan 1024. Selanjutnya, keunggulan performa Cluster pada beban kerja berat di ukuran matriks 1536 dan 2048.

Tabel 6. Uji beda metrik penggunaan memori

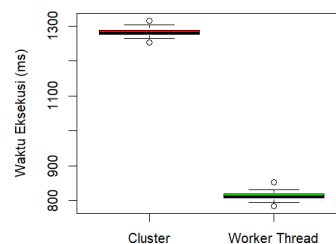
n	p-value	Kesimpulan
512	2,2E-16	Terdapat perbedaan signifikan
1024	2,2E-16	Terdapat perbedaan signifikan
1536	2,2E-16	Terdapat perbedaan signifikan
2048	2,2E-16	Terdapat perbedaan signifikan

Tabel 6 menunjukkan hasil nilai *p-value* di seluruh skenario pengujian tercatat 2,2E-16, yang membuktikan bahwa terdapat perbedaan yang sangat signifikan secara statistik dalam hal penggunaan memori. Perbedaan signifikan ini mengarah pada keunggulan mutlak Worker Thread, yang terbukti jauh lebih efisien dalam penggunaan memori dibandingkan Cluster di semua skenario ukuran matriks yang diuji.

4.2.3 Visualisasi dan Interpretasi Data

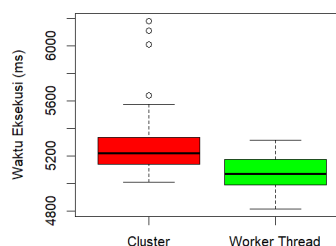
Bagian ini menyajikan visualisasi data dalam bentuk *boxplot* untuk menunjukkan distribusi performa dan perbedaan hasil secara grafis pada setiap ukuran matriks yang diuji.

4.2.3.1 Waktu Eksekusi



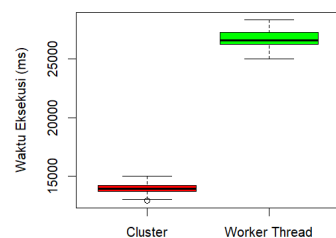
Gambar 2. Boxplot waktu eksekusi matriks 512

Gambar 2 menunjukkan Worker Thread memiliki nilai median yang jauh lebih rendah dibandingkan Cluster. Hal ini menunjukkan bahwa Worker Thread mampu mengeksekusi perhitungan lebih cepat pada beban komputasi ringan. Nilai *p-value* $< 0,05$ dari uji Mann-Whitney mengonfirmasi perbedaan signifikan secara statistik antara kedua modul.



Gambar 3. Boxplot waktu eksekusi matriks 1024

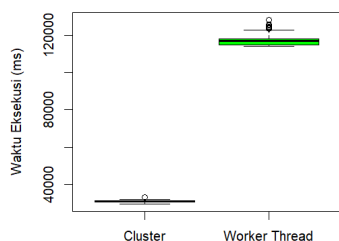
Gambar 3 menunjukkan Worker Thread masih unggul dengan distribusi waktu yang lebih rendah dibandingkan Cluster. Meskipun perbedaannya mulai menyempit, hasil uji Mann-Whitney tetap menghasilkan *p-value* $< 0,05$ yang menandakan adanya perbedaan signifikan secara statistik.



Gambar 4. Boxplot waktu eksekusi matriks 1536

Gambar 4 menunjukkan pembalikan performa di mana Cluster memiliki median waktu yang lebih rendah. Worker Thread mengalami peningkatan waktu eksekusi yang signifikan sehingga menunjukkan penurunan efisiensi pada beban berat. Nilai *p-value* $< 0,05$

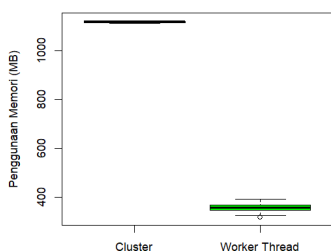
menandakan perbedaan performa yang signifikan.



Gambar 5. Boxplot waktu eksekusi matriks 2048

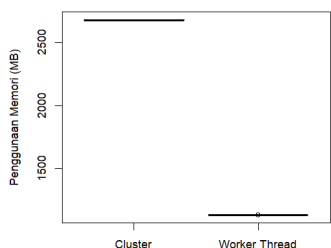
Gambar 5 menunjukkan Cluster semakin unggul, dengan perbedaan jarak median terhadap Worker Thread yang sangat besar. Hasil uji Mann-Whitney kembali mendukung perbedaan signifikan secara statistik dengan $p\text{-value} < 0,05$.

4.2.3.1 Penggunaan Memori



Gambar 6. Boxplot penggunaan memori matriks 512

Gambar 6 menunjukkan Worker Thread jauh lebih efisien dalam penggunaan memori dibandingkan Cluster. Median yang lebih rendah menunjukkan kebutuhan memori yang jauh lebih sedikit dengan $p\text{-value} < 0,05$ menandakan perbedaan signifikan.



Gambar 7. Boxplot penggunaan memori matriks 1024

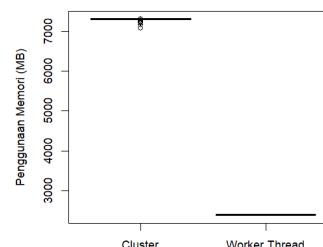
Pada Gambar 7, Worker Thread tetap menunjukkan performa yang konsisten dalam efisiensi memori pada matriks 1024. Perbedaan penggunaan memori relatif besar dibandingkan Cluster, dan $p\text{-value} < 0,05$ memastikan

signifikansi statistik.



Gambar 8. Boxplot penggunaan memori matriks 1536

Gambar 8 menunjukkan Worker Thread lebih hemat memori secara konsisten, dengan selisih yang cukup besar dari Cluster. Perbedaan ini juga signifikan berdasarkan $p\text{-value} < 0,05$.



Gambar 9. Boxplot penggunaan memori matriks 2048

Gambar 9 menunjukkan Cluster menggunakan memori hampir tiga kali lipat dibandingkan Worker Thread. Uji Mann-Whitney menghasilkan $p\text{-value} < 0,05$, mengonfirmasi perbedaan signifikan.

4.3. Pembahasan dan Diskusi

Analisis data kuantitatif menghasilkan tiga temuan statistik utama terkait performa Worker Thread dan Cluster. Pertama, Worker Thread unggul dalam waktu eksekusi pada beban kerja ringan (matriks ukuran 512 dan 1024), namun terjadi pembalikan performa drastis pada beban kerja berat (matriks ukuran 1536 dan 2048) di mana Cluster menjadi lebih cepat. Selain itu, Worker Thread terbukti secara konsisten jauh lebih efisien dalam penggunaan memori dibandingkan Cluster di seluruh skenario pengujian.

Perbedaan performa tersebut dikarenakan pada arsitektur dan mekanisme komunikasi data masing-masing modul. Efisiensi memori Worker Thread disebabkan oleh model *multi-threading* yang berbagi ruang memori, berbeda dengan Cluster yang membutuhkan memori

besar karena isolasi proses penuh. Terkait waktu eksekusi, Cluster terhambat oleh *overhead* komunikasi IPC pada beban ringan, namun menjadi unggul pada beban berat karena *scalability* yang lebih baik dalam menangani data besar. Sebaliknya, penurunan performa Worker Thread pada data besar disebabkan oleh buruknya *scalability* mekanisme *Structured Clone Algorithm* saat mentransfer data matriks secara utuh.

Temuan ini melengkapi penelitian sebelumnya oleh (Patrou et al., 2019) dengan mengisi celah mengenai dampak biaya komunikasi data. Berbeda dengan studi sebelumnya yang menggunakan kasus uji minim komunikasi, penelitian ini membuktikan bahwa pada kasus uji dengan transfer data intensif, arsitektur *multi-process* (Cluster) dapat mengungguli *multi-threaded* (Worker Thread). Hal ini menegaskan bahwa mekanisme pengiriman data dan *overhead* komunikasi merupakan faktor krusial yang mampu membalikkan keunggulan performa komputasi.

5. Penutup

Penelitian ini menunjukkan bahwa tidak ada satu metode paralelisasi yang unggul dalam semua aspek. Pada metrik waktu eksekusi, Worker Thread secara signifikan lebih cepat untuk beban kerja ringan hingga menengah (matriks 512 dan 1024), sedangkan Cluster jauh lebih cepat pada beban berat hingga kompleks (matriks 1536 dan 2048). Sementara itu, dalam hal penggunaan memori, Worker Thread secara konsisten jauh lebih efisien di semua skenario. Temuan ini mengungkap adanya *trade-off* antara kecepatan komputasi dan efisiensi memori, sehingga pemilihan strategi paralelisasi harus disesuaikan dengan karakteristik beban kerja dan ketersediaan sumber daya sistem. Worker Thread ideal untuk tugas ringan atau aplikasi yang memiliki keterbatasan alokasi memori, sedangkan Cluster lebih tepat untuk komputasi sangat berat dengan alokasi memori yang memadai.

Penelitian ini dapat dikembangkan lebih lanjut melalui beberapa arah. Pertama, uji kedua modul pada beban *CPU-bound* lain seperti pemrosesan gambar, kompresi data, atau operasi kriptografi, termasuk skenario dunia nyata seperti pembuatan *thumbnail* secara paralel. Kedua, lakukan pengujian di lingkungan produksi seperti Docker atau platform *cloud* (AWS, GCP, Azure) untuk mengevaluasi

dampak virtualisasi, jaringan, dan alokasi CPU dinamis. Terakhir, eksplorasi optimisasi lebih dalam, seperti penerapan strategi *load balancing* kustom pada Cluster atau penggunaan *SharedArrayBuffer* pada Worker Thread guna mengurangi *overhead* komunikasi dan meningkatkan efisiensi komputasi, terutama pada beban berat.

6. DAFTAR PUSTAKA

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. dan Stein, C., 2022. *Introduction to Algorithms, fourth edition*. MIT Press.
- Dobre, C. dan Xhafa, F., 2014. Parallel Programming Paradigms and Frameworks in Big Data Era. *International Journal of Parallel Programming*, 42(5), pp.710–738. <https://doi.org/10.1007/s10766-013-0272-7>.
- Gascón, U., 2024. *Node.js for Beginners: A comprehensive guide to building efficient, full-featured web applications with Node.js*. Packt Publishing Ltd.
- Node.js, 2025a. *Cluster | Node.js v24.1.0 Documentation*. [online] Available at: <<https://nodejs.org/api/cluster.html>> [Diakses 9 Juni 2025].
- Node.js, 2025b. *Node.js — Introduction to Node.js*. [online] Available at: <<https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>> [Diakses 13 Mei 2025].
- Node.js, 2025c. *Worker threads | Node.js v24.1.0 Documentation*. [online] Available at: <https://nodejs.org/api/worker_threads.html> [Diakses 9 Juni 2025].
- Patrou, M., Kent, K.B. dan Dawson, M., 2019. Scaling Parallelism Under CPU - Intensive Loads in Node.js. In: *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. [online] 2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP). pp.205–210. <https://doi.org/10.1109/EMPDP.2019.8671573>.
- Tilkov, S. dan Vinoski, S., 2010. Node.js: Using JavaScript to Build High-Performance

Network Programs. *IEEE Internet Computing*, 14(6), pp.80–83.
<https://doi.org/10.1109/MIC.2010.145>.